bonus

February 14, 2026

0.1 Bonus Exercise - Performance Analysis and Optimization of the Game of Life Code

0.1.1 Task B.1

Formatting

[2]:
```
!pylint conway.py
```

```
************* Module conway
conway.py:11:31: C0303: Trailing whitespace (trailing-whitespace)
conway.py:24:38: C0303: Trailing whitespace (trailing-whitespace)
conway.py:25:38: C0303: Trailing whitespace (trailing-whitespace)
conway.py:59:28: C0303: Trailing whitespace (trailing-whitespace)
conway.py:64:70: C0303: Trailing whitespace (trailing-whitespace)
conway.py:66:62: C0303: Trailing whitespace (trailing-whitespace)
conway.py:67:62: C0303: Trailing whitespace (trailing-whitespace)
conway.py:68:74: C0303: Trailing whitespace (trailing-whitespace)
conway.py:95:0: C0303: Trailing whitespace (trailing-whitespace)
conway.py:100:0: C0303: Trailing whitespace (trailing-whitespace)
conway.py:127:18: C0303: Trailing whitespace (trailing-whitespace)
conway.py:80:0: R1707: Disallow trailing comma tuple (trailing-comma-tuple)
conway.py:9:0: C0410: Multiple imports on one line (sys, argparse) (multiple-
imports)
conway.py:12:0: R0402: Use 'from matplotlib import animation' instead (consider-
using-from-import)
conway.py:18:0: C0103: Function name "randomGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:18:15: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:22:0: C0103: Function name "addGlider" doesn't conform to snake_case
naming style (invalid-name)
conway.py:29:0: C0103: Function name "addGosperGliderGun" doesn't conform to
snake_case naming style (invalid-name)
conway.py:57:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:57:11: C0103: Argument name "frameNum" doesn't conform to snake_case
naming style (invalid-name)
conway.py:57:32: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
```

```
conway.py:60:4: C0103: Variable name "newGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:57:11: W0613: Unused argument 'frameNum' (unused-argument)
conway.py:83:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:97:4: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:99:8: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:102:4: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:104:8: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:9:0: W0611: Unused import sys (unused-import)

------------------------------------
Your code has been rated at 6.18/10
```

[3]:
```
!black conway.py
```

```
reformatted conway.py

All done!
1 file reformatted.
```

[4]:
```
!pylint conway.py
```

```
************* Module conway
conway.py:9:0: C0410: Multiple imports on one line (sys, argparse) (multiple-
imports)
conway.py:12:0: R0402: Use 'from matplotlib import animation' instead (consider-
using-from-import)
conway.py:19:0: C0103: Function name "randomGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:19:15: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:24:0: C0103: Function name "addGlider" doesn't conform to snake_case
naming style (invalid-name)
conway.py:30:0: C0103: Function name "addGosperGliderGun" doesn't conform to
snake_case naming style (invalid-name)
conway.py:59:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:59:11: C0103: Argument name "frameNum" doesn't conform to snake_case
naming style (invalid-name)
conway.py:59:32: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:62:4: C0103: Variable name "newGrid" doesn't conform to snake_case
naming style (invalid-name)
```

```
conway.py:59:11: W0613: Unused argument 'frameNum' (unused-argument)
conway.py:95:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:111:4: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:113:8: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:116:4: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:118:8: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:9:0: W0611: Unused import sys (unused-import)

------------------------------------------------------------------
Your code has been rated at 7.76/10 (previous run: 6.18/10, +1.58)
```

Documentation

[41]:
```
!mkdir -p docs
!sphinx-quickstart docs -q -p "Bonus" -a "Pavel & Ermia" -v "0" --ext-autodoc
```

```
Finished: An initial directory structure has been created.

You should now populate your master file
/home/pavel/IdeaProjects/DD2358/Ass2/docs/index.rst and create other
documentation
source files. Use the Makefile to build the docs, like so:
   make builder
where "builder" is one of the supported builders, e.g. html, latex or linkcheck.
```

Update `conf.py` with

```
import os
import sys
sys.path.insert(0, os.path.abspath('../..'))
```

to handle being in a separate directory.

[42]:
```
!echo "\nimport os\nimport sys\nsys.path.insert(0, os.path.abspath('../..'))"␣
↪>> docs/conf.py
```

Tell sphinx to autodoc our `conway.py`

[43]:
```
!echo "Conway Module\n=============\n\n.. automodule:: conway\n    :members:\n ␣
↪  :undoc-members:\n    :show-inheritance:" > docs/conway.rst
```

[47]:
```
!echo "   conway" >> docs/index.rst
```

[50]:
```
!make -C docs html
```

```
make: Entering directory '/home/pavel/IdeaProjects/DD2358/Ass2/docs'
Running Sphinx v9.1.0
loading translations [en]… done
loading pickled environment… done
building [mo]: targets for 0 po files that are out of date
writing output…
building [html]: targets for 1 source files that are out of date
updating environment: 0 added, 1 changed, 0 removed
reading sources… [100%] index
looking for now-outdated files… none found
pickling environment… done
checking consistency… done
preparing documents… done
copying assets…
copying static files…
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/basic.css
Writing evaluated template result to /home/pavel/IdeaProjects/DD2358/Ass2/docs/_
build/html/_static/documentation_options.js
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/language_data.js
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/alabaster.css
copying static files: done
copying extra files…
copying extra files: done
copying assets: done
writing output… [100%] index
generating indices… genindex py-modindex done
writing additional pages… search done
dumping search index in English (code: en)… done
dumping object inventory… done
build succeeded.

The HTML pages are in _build/html.
make: Leaving directory '/home/pavel/IdeaProjects/DD2358/Ass2/docs'
```

[51]:
```
from IPython.display import HTML, display

display(HTML(filename="docs/_build/html/conway.html"))
```

```
<IPython.core.display.HTML object>
```

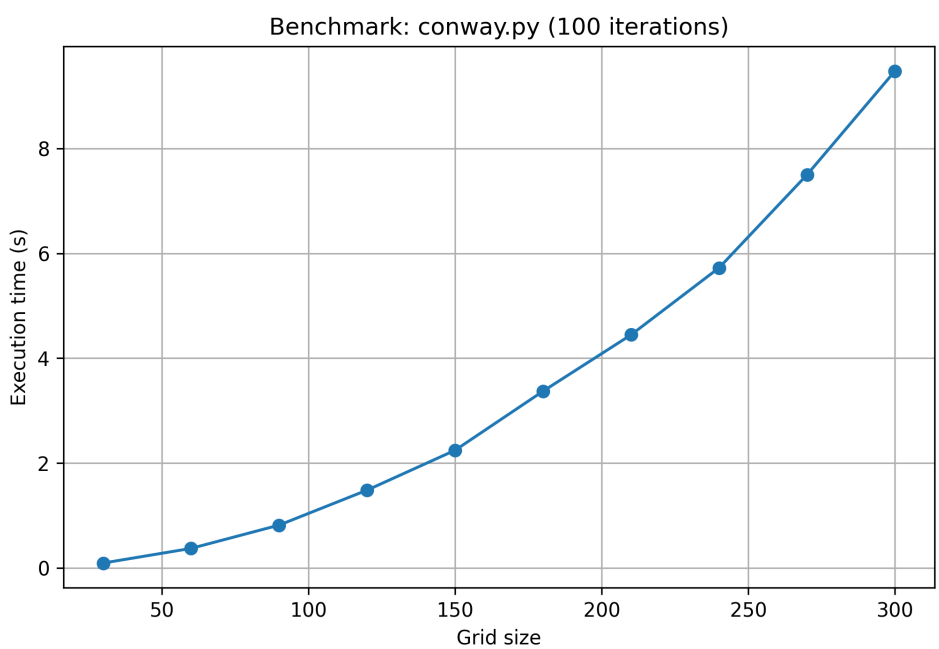
0.1.2 Task B.2

From this point on part the source code was modified to disable the animation and make benchmarking easier.

[53]:
```python
# %load benchmark.py
import conway
import timeit
import matplotlib.pyplot as plt

grid_sizes = [30, 60, 90, 120, 150, 180, 210, 240, 270, 300]

time = []
for size in grid_sizes:
    t1 = timeit.default_timer()
    conway.main(size)
    t2 = timeit.default_timer()
    time.append((t2 - t1))

plt.figure(figsize=(8, 5))
plt.plot(grid_sizes, time, marker='o')
plt.xlabel("Grid size")
plt.ylabel("Execution time (s)")
plt.title(f"Benchmark: conway.py (100 iterations)")
plt.grid(True)
plt.savefig("benchmark.png", dpi=300)
plt.show()
```

[54]:
```python
from IPython.display import Image
Image("benchmark.png")
```

[54]:



As can be seen the growth is quadratic as is the grid size N^2.

0.1.3 Task B.3

[55]:
```
!python -m cProfile -o p.stats conway.py
```

[ ]:
```
import pstats
stats = pstats.Stats("p.stats")
stats.sort_stats("cumulative")
stats.print_stats()
```

Most of the time is spent in the `update` function so gonna focus attention there.

[60]:
```
!python -m kernprof -l -v conway.py
```

```
Wrote profile results to 'conway.py.lprof'
Timer unit: 1e-06 s

Total time: 7.59332 s
File: conway.py
Function: update at line 58

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    58                                           @profile
    59                                           def update(frameNum, img, grid,
N):
    60                                               # copy grid since we
require 8 neighbors for calculation
    61                                               # and we go line by line
    62       100        491.2      4.9      0.0      newGrid = grid.copy()
    63     10100       2758.5      0.3      0.0      for i in range(N):
    64   1010000     276403.1      0.3      3.6          for j in range(N):
    65                                                       # compute 8-neghbor
sum
    66                                                       # using toroidal
boundary conditions - x and y wrap around
    67                                                       # so that the
simulaton takes place on a toroidal surface.
    68   2000000     613692.7      0.3      8.1              total = int(
    69   1000000     312319.6      0.3      4.1                  (
    70   8000000    2722387.2      0.3     35.9                      grid[i, (j
- 1) % N]
    71   1000000     363180.0      0.4      4.8                      + grid[i,
(j + 1) % N]
    72   1000000     386949.0      0.4      5.1                      + grid[(i -
1) % N, j]
    73   1000000     379234.7      0.4      5.0                      + grid[(i +
1) % N, j]
```

```
    74   1000000     396301.1      0.4      5.2                     + grid[(i -
1) % N, (j - 1) % N]
    75   1000000     383924.7      0.4      5.1                     + grid[(i -
1) % N, (j + 1) % N]
    76   1000000     390978.8      0.4      5.1                     + grid[(i +
1) % N, (j - 1) % N]
    77   1000000     383196.9      0.4      5.0                     + grid[(i +
1) % N, (j + 1) % N]
    78                                                             )
    79   1000000     259407.8      0.3      3.4                     / 255
    80                                                         )
    81                                                         # apply Conway's
rules
    82   1000000     413692.2      0.4      5.4                 if grid[i, j] ==
ON:
    83    103960      29584.6      0.3      0.4                     if (total < 2)
or (total > 3):
    84     42425      16544.1      0.4      0.2                         newGrid[i,
j] = OFF
    85                                                         else:
    86    896040     244449.0      0.3      3.2                     if total == 3:
    87     41136      16176.9      0.4      0.2                         newGrid[i,
j] = ON
    88                                                 # update data
    89                                                 #img.set_data(newGrid)
    90       100       1387.6     13.9      0.0       grid[:] = newGrid[:]
    91       100        256.1      2.6      0.0       return (img,)
```

Calculating the neighbor sum takes the majority of the time.

0.1.4 Task B.4

By applying vectorization, using the `roll` function, we can take the neighboring sum outside of the loop and gain performance that way. Next step would be to vectorize the creation of the `newGrid` and apply inplace operations.

[ ]:
```
def update_old(frameNum, img, grid, N):
    #...
    newGrid = grid.copy()
    for i in range(N):
        for j in range(N):
            total = int(
                (
                        grid[i, (j - 1) % N]
                        + grid[i, (j + 1) % N]
                        + grid[(i - 1) % N, j]
                        + grid[(i + 1) % N, j]
```

```
                        + grid[(i - 1) % N, (j - 1) % N]
                        + grid[(i - 1) % N, (j + 1) % N]
                        + grid[(i + 1) % N, (j - 1) % N]
                        + grid[(i + 1) % N, (j + 1) % N]
                )
                / 255
            )
    #...
```

[ ]:
```
import numpy as np
def update_new(frameNum, img, grid, N):
    #...
    top = np.roll(grid, 1, 0)
    bottom = np.roll(grid, -1, 0)

    total2 = (top + bottom
              + np.roll(grid, 1, 1)
              + np.roll(grid, -1, 1)
              + np.roll(top, 1, 1)
              + np.roll(top, -1, 1)
              + np.roll(bottom, 1, 1)
              + np.roll(bottom, -1, 1)
              ) / 255

    for i in range(N):
        for j in range(N):
            total = total2[i][j]
    #...
```

[ ]:
```
!python -m cProfile conway2.py
```

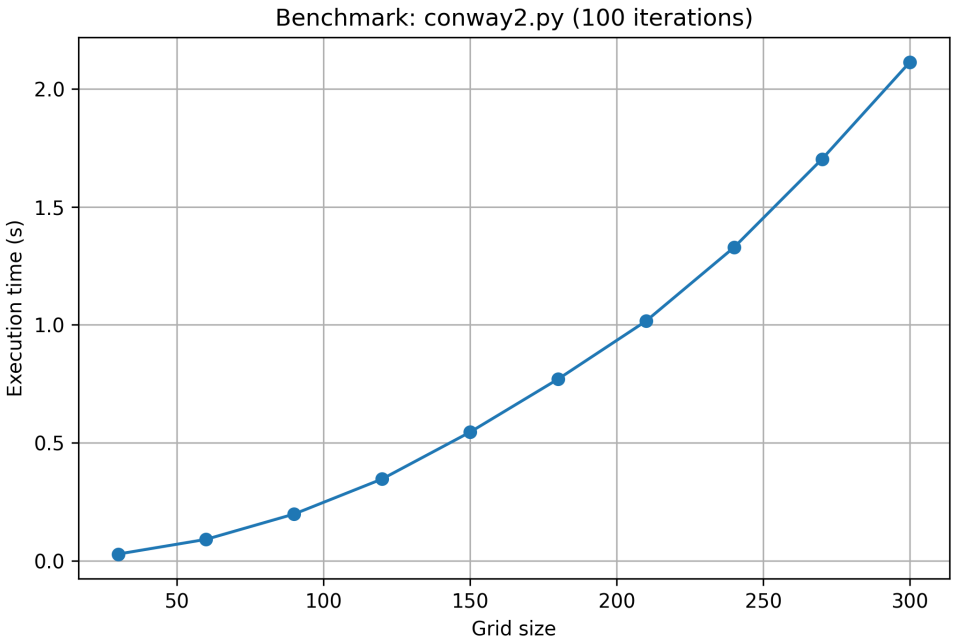
[4]:
```
!python -m kernprof -l -v conway2.py
```

```
Wrote profile results to 'conway2.py.lprof'
Timer unit: 1e-06 s

Total time: 1.4806 s
File: conway2.py
Function: update at line 59

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    59                                           @profile
    60                                           def update(frameNum, img, grid,
N):
    61                                               # copy grid since we
require 8 neighbors for calculation
    62                                               # and we go line by line
```

```
    63       100        433.9      4.3      0.0      newGrid = grid.copy()
    64
    65       100       4443.1     44.4      0.3      top = np.roll(grid, 1, 0)
    66       100       2458.1     24.6      0.2      bottom = np.roll(grid, -1,
0)
    67
    68       800       4572.3      5.7      0.3      total2 = (top + bottom
    69       100       2453.9     24.5      0.2               + np.roll(grid,
1, 1)
    70       100       2140.9     21.4      0.1               + np.roll(grid,
-1, 1)
    71       100       2146.6     21.5      0.1               + np.roll(top, 1,
1)
    72       100       2122.3     21.2      0.1               + np.roll(top,
-1, 1)
    73       100       2095.3     21.0      0.1               + np.roll(bottom,
1, 1)
    74       100       2096.9     21.0      0.1               + np.roll(bottom,
-1, 1)
    75       100         36.8      0.4      0.0               ) / 255
    76
    77     10100       2655.0      0.3      0.2      for i in range(N):
    78   1010000     279656.4      0.3     18.9          for j in range(N):
    79                                                         # compute 8-neghbor
sum
    80                                                         # using toroidal
boundary conditions - x and y wrap around
    81                                                         # so that the
simulaton takes place on a toroidal surface.
    82
    83   1000000     436375.3      0.4     29.5              total =
total2[i][j]
    84                                                         # apply Conway's
rules
    85   1000000     400489.8      0.4     27.0              if grid[i, j] ==
ON:
    86    120129      39637.3      0.3      2.7                  if (total < 2)
or (total > 3):
    87     49651      18532.2      0.4      1.3                      newGrid[i,
j] = OFF
    88                                                         else:
    89    879871     259567.2      0.3     17.5                  if total == 3:
    90     48565      17757.8      0.4      1.2                      newGrid[i,
j] = ON
    91                                                 # update data
    92                                                 #img.set_data(newGrid)
    93       100        759.8      7.6      0.1      grid[:] = newGrid[:]
    94       100        169.8      1.7      0.0      return (img,)
```

[6]:
```python
from IPython.display import Image
Image("benchmark2.png")
```

[6]:



As can be seen the optimization was a success, the correctness was tested with `assert` against the original implementation. The performance gains are significant as now we can run a 300x300 grind in about the same time it took for a 150x150 grid with the old implementation.